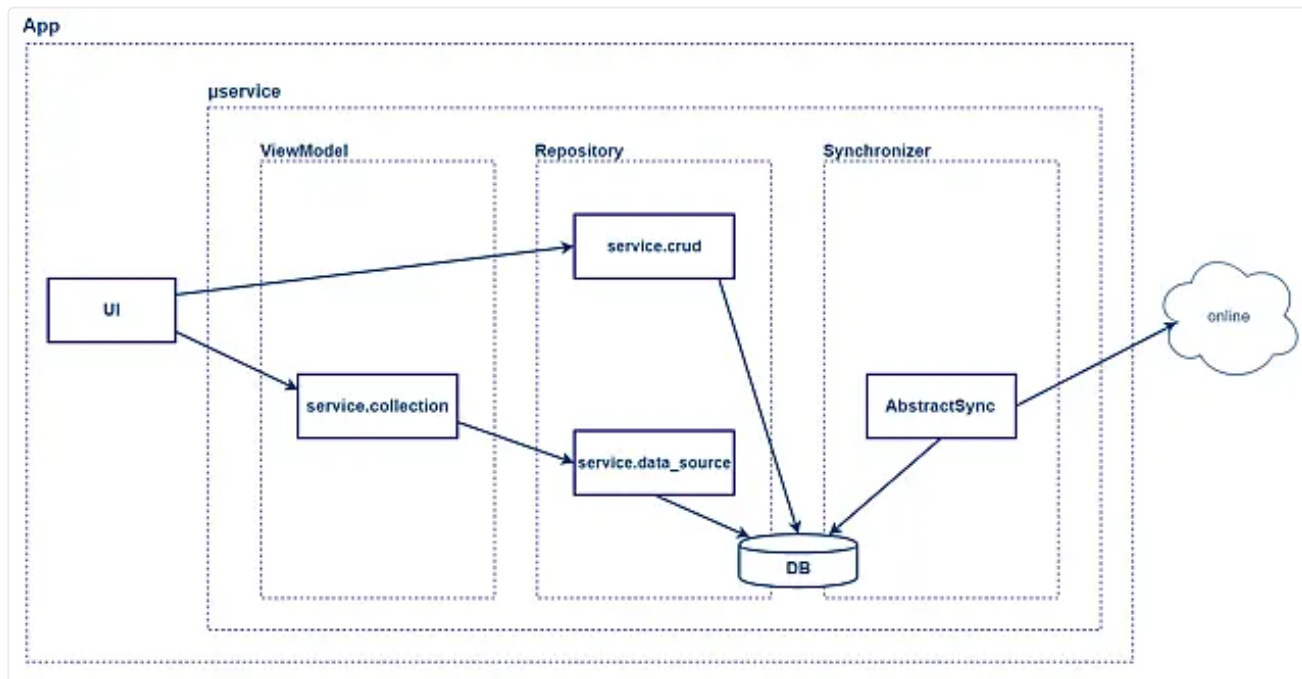


Профилирование запросов SQLite

Описание слоев приложения

Наши [desktop и мобильные приложения](#) разбиты на слои, каждый из которых занят своей задачей - обслуживание своего потребителя. Наглядно это можно отобразить на следующей схеме:



1. **ViewModel** - слой, предназначенный для работы с потребителем.
2. **Repository** - слой, выступающий в качестве источника данных для слоя ViewModel. Из слоя Repository пользователь может создавать, читать, модифицировать и удалять данные.
3. **Synchronizer** - слой, предназначенный для обмена данными между локальным кэшем и облаком.
4. **База данных (DB)** - SQLite - это связующий слой между слоями Repository и Synchronizer. База данных является основой **асинхронного размыкания**, обеспечивающего ощущение реактивности и отзывчивости нашего приложения, а также выполняет функции постоянного хранилища данных микросервиса. Слой Synchronizer **пишет данные в базу данных**, Repository - **читает их из базы данных**, и наоборот.

База данных является одним из важных слоев нашей системы и она, как и другие слои, подвержена различным проблемам производительности.

Проблемы производительности БД

Производительность БД может снижаться вследствие следующих причин:

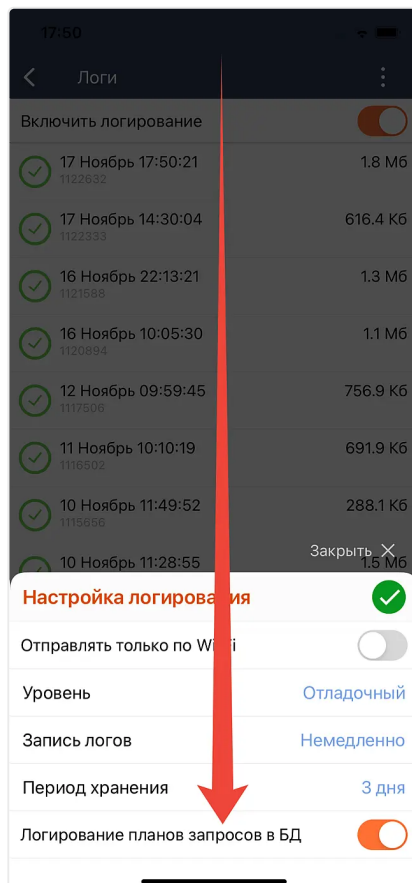
- отсутствие индекса
- проблемы дисковой производительности
- недостаток мощности процессора
- неудачная схема БД для запрашиваемого набора данных
- рекурсивные запросы
- большой объем данных, которые необходимо сгруппировать
- нехватка объема памяти на диске

Для того чтобы решить проблемы производительности запросов к БД обычно инспектируют планы запросов на реальных данных. Для этого у нас есть система, которая позволяет увидеть эти данные с помощью логирования плана запросов.

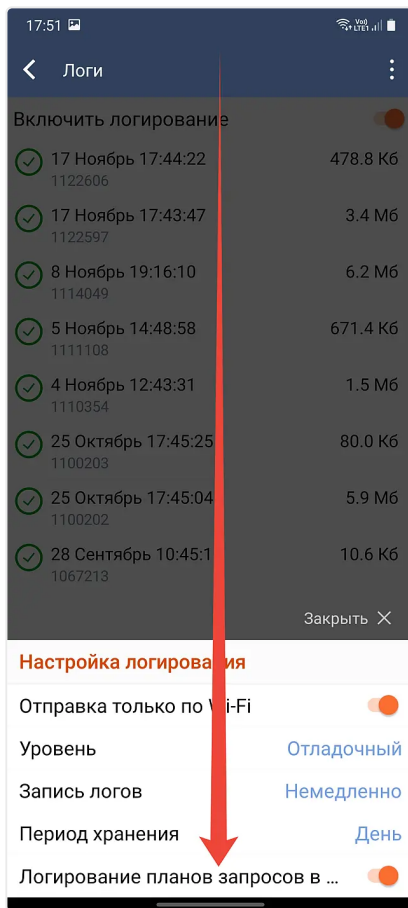
Данный режим не подходит для штатной работы приложения, потому что он делает выполнение запроса с планом, а потом, по сути, второй раз выполняет запрос. Исключение: "точечная" диагностика запросов с последующим отключением.

Как включить логирование плана запросов

- Через параметры контроллера приложения:
 - 1 **[Ядро.База данных.SQLite]**
 - 2 `ЛогироватьПланЗапроса=true`
- Из инженерного экрана iOS:



- Из инженерного экрана Android:



Логирование планов запросов в SQLite

Команда SQLite [EXPLAIN QUERY PLAN](#) может рассказать, что происходит, когда выполняется запрос. Логирование планов запросов и статистики выполнения запроса даёт возможность обнаружить медленные/проблемные запросы. Корректировка таких запросов позволит сократить время, затрачиваемое на обработку запроса, что впоследствии может значительно ускорить работу микросервиса/приложения.

После включения SQL-запросы в логах имеют дополнительную информацию: план запроса и статистика выполнения запроса.

```
1 SELECT * FROM FM_Selling WHERE FieldworkUuid = $1
2 plan:
3 --SCAN TABLE FM_Selling
4
5 [sql][stats]
6 Rows Queried: 2
7 Fullscan Steps: 34
8 Sort Operations: 0
9 Autoindex Inserts: 0
10 Virtual Machine Steps: 124
11 Memory Used (bytes): 5392
```

The screenshot shows the 'Управление облаком' (Cloud Management) interface. On the left is a sidebar with navigation options like 'Структура облака', 'Анализ работы', 'Статистика', 'Журнал логов', 'Журнал вызовов', 'Отчет по ошибкам', 'Планы вызовов', 'Видеообзор', 'Неиспользуемые вызовы', 'Утечки памяти', 'Профилирование', 'Очередь запросов', 'Выделенные ресурсы', 'Анализ входов', 'Настройки', 'Ключи', and 'Пользователи облака'. The main area displays a table of queries with columns for 'Время', 'Полное имя', 'IP Вызовов', 'Выполнено', 'План', 'Тип', 'Уptime', 'План метода', and 'События'. A red arrow points to the 'Fullscan Steps' field in the statistics section of a query, which has a value of 34.

🔍 Explain-режим диагностирования sql-запросов дает дополнительный накладной расход, и не может использоваться при регулярной работе приложения, только для "точной" диагностики с последующим отключением.

Значение полей статистики

- Fullscan Steps - количество раз, которое [движок байт-кода](#) прошел через всю таблицу. Если это поле не нулевое, то возможно стоит добавить индекс.
- Sort Operations - количество раз, которое движок байт-кода должен был отсортировать записи. Положительное значение в этом поле может указывать на возможность повышения производительности путем добавления индекса.
- Autoindex Inserts - поле содержит количество строк, которые были вставлены в [автоматические индексы](#) (не путать с внутренними). Если это поле не 0, то **обязательно** надо добавить индекс.
- Virtual Machine Steps - это поле представляет собой количество операций механизма байт-кода, которые были выполнены. Это поле можно использовать в качестве прокси для того, сколько процессорного времени использовал оператор.
- Memory Used (bytes) - количество байт памяти, которое было использовано для выполнения запроса.

Более подробно о значениях полей статистики можно почитать [здесь](#).

Что может указать на проблемы в запросе?

Про индексы, как они работают и в каких случаях их лучше использовать можно почитать в [документации SQLite](#).

Fullscan Steps

В запросе выше, в таблице FM_Selling не было индекса и значение поля FullscanSteps = 34. Добавив индекс по полю FieldworkUuid (т.к. в запросе фильтрация происходит по этому полю) план запроса и статистика запроса изменились.

```

1 SELECT * FROM FM_Selling WHERE FieldworkUuid = $1
2 plan:
3 --SEARCH TABLE FM_Selling USING INDEX iFM_Selling-pFieldwork-
  FieldworkUuid (FieldworkUuid=?)
4
5 [sql][stats]
6 Rows Queried: 2

```

```

7 Fullscan Steps: 0
8 Sort Operations: 0
9 Autoindex Inserts: 0
10 Virtual Machine Steps: 27
11 Memory Used (bytes): 5520

```

Virtual Machine Steps без индекса	Virtual Machine Steps с индексом
124	27

Как можно заметить, количество шагов виртуальной машины при запросе с индексом уменьшилось в 4,7 раза.

Automatic index

Когда нет доступных индексов, SQLite может создать [автоматический индекс](#), который существует только на время выполнения одного запроса SQL. SQLite предполагает, что данных так много, что создание временного индекса дешевле, чем поиск без индекса. Не путайте автоматические индексы с [внутренними](#) (internal) индексами (имеющие имена, как `sqlite_autoindex_table_N`). Автоматические индексы существуют только на время выполнения одного запроса, никогда не сохраняются на диске и видны только при одном подключении к базе данных.

Пример автоматического индекса:

```

1 SELECT * FROM FM_DeliveryTask AS t
2     INNER JOIN FM_Fieldwork AS f ON f.Uuid = t.Uuid
3     INNER JOIN Document as doc ON f.uuid = doc.uuid
4     WHERE (f.DateStart >= $1 OR f.DateEnd >= $1) AND
5           (f.DateStart <= $2 OR f.DateEnd <= $2) AND
6           (doc.state = 10 AND doc.deleted = 0) AND
7           (f.IsCurrentExecutor = $3)
8     ORDER BY f.DateStart
9 plan:
10 --SCAN TABLE FM_Fieldwork AS f USING INDEX iFM_Fieldwork-pOrder-Dates
11 --SEARCH TABLE Document AS doc USING INDEX iDocument-Uuid (Uuid=?)
12 --SEARCH TABLE FM_DeliveryTask AS t USING AUTOMATIC COVERING INDEX
   (Uuid=?)
13
14 [sql][stats]
15 Rows Queried: 0
16 Fullscan Steps: 343
17 Sort Operations: 0
18 Autoindex Inserts: 189
19 Virtual Machine Steps: 3067
20 Memory Used (bytes): 36280

```

В данном примере в таблице `FM_DeliveryTask` нет индекса и оптимизатор SQLite решил, что выгоднее добавить свой для этого запроса. Если уж SQLite создала индекс, то это однозначно надо сделать и разработчику. Запрос после добавления индекса по полю `Uuid` в таблице `FM_DeliveryTask`.

```

1 SELECT * FROM FM_DeliveryTask AS t
2     INNER JOIN FM_Fieldwork AS f ON f.Uuid = t.Uuid
3     INNER JOIN Document as doc ON f.uuid = doc.uuid
4     WHERE (f.DateStart >= $1 OR f.DateEnd >= $1) AND
5           (f.DateStart <= $2 OR f.DateEnd <= $2) AND
6           (doc.state = 10 AND doc.deleted = 0) AND
7           (f.IsCurrentExecutor = $3)
8     ORDER BY f.DateStart
9 plan:
10 --SEARCH TABLE Document AS doc USING INDEX iDocument-State (State=?)
11 --SEARCH TABLE FM_Fieldwork AS f USING INDEX

```

```

12  sqlite_autoindex_FM_Fieldwork_1      (Uuid=?)
    --SEARCH TABLE FM_DeliveryTask AS t USING COVERING INDEX
    sqlite_autoindex_FM_DeliveryTask_1 (Uuid=?)
13  `--USE TEMP B-TREE FOR ORDER BY
14
15  [sql][stats]
16  Rows Queried: 0
17  Fullscan Steps: 0
18  Sort Operations: 1
19  Autoindex Inserts: 0
20  Virtual Machine Steps: 1886
21  Memory Used (bytes): 36336

```

Virtual Machine Steps без индекса	Virtual Machine Steps с индексом
3067	1886

Количество шагов виртуальной машины снизилось почти на 40%.

Temp B-Tree

Если запрос SELECT содержит ORDER BY, GROUP BY или DISTINCT, SQLite может использовать временную структуру b-дерева для сортировки выходных строк.

Пример запроса и плана запроса:

```

1  SELECT * FROM FM_Address
2      WHERE FieldworkUuid = $1
3      ORDER BY Type
4  plan:
5  |--SEARCH TABLE FM_Address USING INDEX iFM_Address-pAddress-
   FieldworkUuid (FieldworkUuid=?)
6  `--USE TEMP B-TREE FOR ORDER BY
7
8  [sql][stats]
9  Rows Queried: 1
10 Fullscan Steps: 0
11 Sort Operations: 1
12 Autoindex Inserts: 0
13 Virtual Machine Steps: 49
14 Memory Used (bytes): 6480

```

Согласно [документации SQLite](#), "использование индекса почти всегда намного эффективнее, чем выполнение сортировки". Добавим составной индекс по полю фильтрации и полю сортировки.

Время исполнения запроса

В прошлых пунктах "скорость" выполнения запроса измерялась в количестве шагов виртуальной машины, которое было выполнено для исполнения запроса. Но стоит знать, что не все шаги виртуальной машины выполняются одинаково долго по времени. Поэтому также стоит смотреть на такой параметр, как время выполнения запроса.

```

1  [sql][start][public]
2  DELETE FROM FM_Selling WHERE FieldworkUuid = $1
3  plan:
4  --SEARCH TABLE FM_Selling USING COVERING INDEX iFM_Selling-pFieldwork-
   FieldworkUuid
5  (FieldworkUuid=?)
6
7  [sql][finish] [exec_period] 0

```

